CÓMPUTO PARALELO Y DISTRIBUIDO · TEMA 5

Memoria compartida distribuida

y modelos de consistencia



10 de noviembre de 2026

Equipo 5

Ever Garcia Ramirez

Christian Luna Ortiz

Marcelino Parra Hernández

José José Varela Palacios

Contenido

01	Qué es la DSM y cómo se simula	Ever
02	Esquemas: páginas, variables y objetos	Christian
03	Modelos de consistencia	Marcelino
04	Acceso del Equipo A a datos del Equipo B	José
05	Prueba real en un clúster de laptops	Christian

01

Qué es la DSM y cómo se simula

Expone: Ever



Dos formas de comunicar procesos

Memoria compartida

- Procesos leen y escriben en un mismo espacio de direcciones.
- Programación simple.
- Requiere hardware común (multiprocesador).

Paso de mensajes (MPI)

- Cada nodo tiene memoria propia.
- Comunicación explícita: `send`/`recv`.
- Escala en clústeres de computadoras separadas.

Pregunta: ¿podemos tener la comodidad de la primera sobre el hardware de la segunda?

Memoria compartida distribuida (DSM)

- **Abstracción**: una capa de software que presenta a los procesos de varias computadoras *un solo espacio de direcciones* compartido.
- Por debajo, los datos viven en la memoria local de cada nodo y viajan por la red cuando se necesitan.

Cómo se simula

1. Se divide el espacio compartido en unidades (páginas, variables u objetos).
2. Se registra qué nodo tiene cada unidad (directorio).
3. Un acceso a una unidad remota se intercepta y se resuelve por la red.
4. Se guardan copias locales y se mantienen coherentes.

Problemas que hay que resolver

- **Latencia**: acceder a un dato remoto cuesta órdenes de magnitud más que uno local.
- **Localización**: ¿quién tiene la copia actual? (directorio central o distribuido).
- **Replicación**: copias múltiples aceleran lecturas, pero complican las escrituras.
- **Coherencia y consistencia**: qué valor ve cada proceso y en qué orden.
- **Falso compartimiento**: dos variables distintas en la misma unidad provocan tráfico innecesario.

02

Esquemas: páginas, variables y objetos

Expone: Christian



Tres granularidades

	Páginas	Variables	Objetos
Unidad	Página de memoria virtual (p. ej. 4 KB)	Variable compartida declarada	Objeto con operaciones
Detección	Falla de página (MMU)	Compilador o biblioteca	Invocación de método
Ejemplos	IVY, TreadMarks	Munin	Orca, Linda (tuplas)
Ventaja	Transparente, sin cambiar el programa	Grano fino, menos falso compartimiento	Encapsula datos y coherencia
Desventaja	Falso compartimiento	Hay que anotar el código	Modelo de programación nuevo

DSM basada en páginas

- Se aprovecha la MMU: las páginas remotas se marcan como *no presentes* o de *solo lectura*.
- Una falla de página dispara el protocolo de DSM, que trae la página y reintenta el acceso.
- Un *dueño* (o gestor) conoce el estado de cada página: lectura compartida o escritura exclusiva.

Coherencia típica

- **Escritura-invalidación**: antes de escribir se invalidan las demás copias.
- **Escritura-actualización**: la escritura se propaga a todas las copias.

DSM basada en variables y en objetos

Variables compartidas (Munin)

- Solo se comparte lo declarado; cada variable usa el protocolo adecuado a su patrón (solo lectura, migratoria, productor-consumidor…).
- Cerrojos y barreras: los cambios se propagan al liberar.

Objetos (Orca, Linda)

- Los datos solo se tocan mediante operaciones; el sistema replica o migra el objeto.
- Operaciones atómicas: consistencia más simple.
- Linda: se intercambian *tuplas* con `out`, `in`, `rd`.

03

Modelos de consistencia

Expone: Marcelino



Coherencia vs. consistencia

- **Coherencia**: qué valores puede devolver una lectura de *una misma* ubicación.
- **Consistencia**: en qué orden se ven las operaciones sobre *distintas* ubicaciones.

Un modelo de consistencia es un contrato

- Más estricto ⇒ más fácil de programar, pero más lento (más mensajes y esperas).
- Más relajado ⇒ más rendimiento, pero el programador sincroniza explícitamente.

Modelos de consistencia



Estricta Toda lectura devuelve la última escritura en tiempo real. Imposible de lograr en una red real.

Secuencial (Lamport) El resultado es el de algún orden secuencial único que respeta el orden de cada proceso; todos ven el mismo orden.

Causal Solo se garantiza el orden de operaciones relacionadas causalmente; las concurrentes pueden verse en distinto orden.

Débil / de liberación El orden solo se garantiza en puntos de sincronización (`acquire`/`release`).

Eventual Si dejan de haber escrituras, todas las réplicas convergen con el tiempo.

Ejemplo: secuencial vs. causal

</> Programa

```
P1: x = 1
P2: if (x == 1) y = 1
P3: lee y, luego x
```

Pregunta: si P3 lee `y == 1`, ¿puede leer después `x == 0`?

- Secuencial: no. Todos ven el mismo orden.
- Causal: no. `y=1` depende causalmente de `x=1`.
- Eventual: sí, momentáneamente.

Comparación

Modelo	Facilidad	Rendimiento	Sincronización
Estricta	Máxima	Muy bajo	No necesaria
Secuencial	Alta	Bajo	No necesaria
Causal	Media	Medio	Parcial
Débil / liberación	Baja	Alto	Explícita
Eventual	Baja	Muy alto	Lógica de aplicación

04

Acceso del Equipo A a datos del Equipo B

Expone: José



Cómo accede el Equipo A a datos del Equipo B



Para el proceso en A todo parece una lectura local de memoria.

Pseudocódigo: lo que hace cada nodo

```
leer(dir):
  p = pagina(dir)
  si p esta en cache: // R o W
     return cache[p][dir]
  enviar(gestor, LEER, p) // falla
  datos = recibir(gestor)
  cache[p] = (R, datos) // copia
  return cache[p][dir]
```

```
escribir(dir, v):
  p = pagina(dir)
  si cache[p].modo != W:
     // pedir propiedad EXCLUSIVA
     enviar(gestor, ESCRIBIR, p)
     datos = recibir(gestor)
     cache[p] = (W, datos)
  cache[p][dir] = v
```

🔒 El nodo nunca escribe sin ser el único dueño: así dos escrituras simultáneas no se pisan.

Pseudocódigo: el gestor (escritura-invalidación)

```
al recibir ESCRIBIR(p) de n:
  si duenio[p] != ninguno y duenio[p] != n:
     datos[p] = traer_e_invalidar(duenio[p], p) // ultima version
  para c en copias[p] - {n}:
     enviar(c, INVALIDA, p); esperar ACK de c // nadie lee valores viejos
  duenio[p] = n; copias[p] = {}
  responder(n, DATOS, datos[p], modo = W)

al recibir LEER(p) de n:
  si duenio[p] != ninguno:
     datos[p] = traer(duenio[p], p) // el dueno baja a modo R
  copias[p] = copias[p] + {n}
  responder(n, DATOS, datos[p], modo = R)
```

05

Prueba real en un clúster de laptops

Expone: Christian



Nuestro clúster: maestro–trabajador



Prototipo propio en Python (solo biblioteca estándar): DSM por páginas de 4 KB, gestor central y escritura-invalidación. IPs ilustrativas; las mediciones de las siguientes diapositivas se hicieron con este mismo programa.

Latencia: el costo de una falla de página

Modelo

$$T_{\text{falla}} \approx T_{\text{RTT}} + \frac{S_{\text{página}}}{B} + T_{\text{gestor}} \qquad \text{penalización} = \frac{T_{\text{falla}}}{T_{\text{local}}}$$

Misma máquina (*loopback*)

$\approx 700\times$

RTT 0.06–0.10 ms · falla 0.21 ms

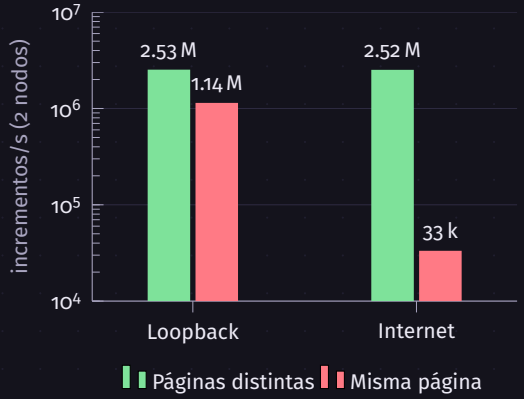
Internet (laptop ↔ servidor, Tailscale)

$\approx 376\,000\times$

RTT 111 ms · falla 113 ms

Veces más lento que una lectura local ($T_{\text{local}} \approx 0{,}30\ \mu$s, página ya en caché, en Python). La falla casi es solo T_{RTT}: la red domina.

Falso compartimiento, medido



Dos trabajadores incrementan *cada uno su propia variable* al mismo tiempo.

- Páginas distintas: o fallas.
- Misma página: la página rebota entre nodos (invalidación tras invalidación).

loopback	internet
2.2×	**76×**

más lento con la misma página. Resultado final correcto en todas las corridas.

Balanceo CPU/GPU con DSM

El dueño calcula (*owner computes*)

- El maestro reparte *páginas*, no tareas sueltas: cada nodo calcula sobre lo que ya tiene.
- Laptops con GPU reciben bloques más grandes, en proporción a su velocidad medida.

Reglas que salen de la medición

- Alinear los datos de cada nodo a *su* página: evita el 76×.
- La VRAM no forma parte de la DSM: el nodo copia su página a la GPU, calcula y escribe de vuelta.
- Consistencia de liberación: enviar cambios en lotes al `release`, no en cada escritura.

Conclusiones

- ✔ La DSM da la comodidad de memoria compartida sobre computadoras con memoria separada.
- ✔ La elección de granularidad (páginas, variables, objetos) equilibra transparencia, falso compartimiento y esfuerzo de programación.
- ✔ El modelo de consistencia es el compromiso central entre facilidad de uso y rendimiento.
- ✔ Lo medimos: en red, una falla cuesta $\sim 10^5$ veces una lectura local; acomodar los datos importa más que el procesador.
- ✔ Por eso en clústeres reales domina el paso de mensajes (MPI): el costo de la red queda explícito.

Referencias

- Tanenbaum, A. y Van Steen, M. *Sistemas distribuidos: principios y paradigmas*.
- Li, K. y Hudak, P. (1989). Memory coherence in shared virtual memory systems. *ACM TOCS*.
- Lamport, L. (1979). How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Trans. Computers*.
- Bennett, J. et al. (1990). Munin: distributed shared memory based on type-specific memory coherence. *PPOPP*.
- Keleher, P. et al. (1994). TreadMarks: distributed shared memory on standard workstations and operating systems. *USENIX*.

¿Preguntas?

Gracias

Equipo 5 · Memoria compartida distribuida

